CORE THEORY USED IN THE PROJECT

📌 Lexicon-Based Sentiment Analysis

Our project uses a **dictionary-based approach (NOT machine learning)**.

- Words are stored like:
  - good → +1
  - bad → -1
- Then compute total score

This method is Simple, fast, explainable.

📌 Text Classification Concept

At its core, your system performs:

Text Classification

- Input → text
- Output → sentiment category

This is one of the main NLP tasks

📌 Tokenization

We convert text into words:

"I love this movie"
→ ["I", "love", "this", "movie"]

📌 Polarity Scoring

We compute a **numerical score**: Score = Σ(word scores)

Then classify:

- 0 → Positive
- <0 → Negative
- =0 → Neutral

🧩 Rule-Based Decision System

Our system is **rule-based**, not learned, which means there is no training data, no neural networks. It's purely logic based.

COMPUTER SCIENCE CONCEPTS USED

🧩 Object-Oriented Programming (OOP)

- **Inheritance**

class Scorer → WordCountScorer, WeightedScorer

Reuse structure, extend behavior

- **Polymorphism**

Scorer* scorer;

Same interface, different implementations

🧩 Data Structures

unordered_map

Used for: Word → score mapping

Provides: **O(1) lookup time (very fast)**

🧩 Memory Management (DMA)

We implemented:

char* data = new char[...]

which shows Low-level memory control, Efficient storage

📌 Move Semantics

Buffer(move(text));

Avoids copying large data
Improves performance

📌 Exception Handling

throw runtime_error(...)

Makes system:

- Safe
- Robust

📌 Templates

Generic programming:

template<typename T>

Reusable logic for multiple data types

COMPLETE SYSTEM ARCHITECTURE

Input File
↓
Buffer (DMA storage)
↓
Tokenizer
↓
Dictionary Lookup (unordered_map)
↓
Scorer (Polymorphism)
↓
Sentiment Score
↓
Output

WHAT IS UNIQUE ABOUT YOUR PROJECT?

Combines NLP + Core C++

Most projects use Python libraries. Our project is built **from scratch in C++**, without any external NLP library.

Dual Scoring Mechanism

We implemented:

- WordCountScorer
- WeightedScorer

Which shows Flexibility, and Comparative analysis .

The application supports **two scoring methods** using **polymorphism**:

Method 1: Word Count Scorer

Logic:

- Counts sentiment based on presence of words
- Ignores intensity (weights)

Rules:

- Positive word → +1
- Negative word → -1

Example:

Input: "This movie is amazing but bad"

amazing → +1

bad → -1

Final Score = 0 → Neutral

❖ **Characteristics:**

- Simple and fast
- Less accurate

- Does not consider word strength

Method 2: Advanced Scorer

Logic:

- Uses actual sentiment weights from dictionary
- Performs basic NLP processing

Features:

1. **Weighted scoring**: Uses actual values (e.g., +5, -2)
2. **Text cleaning**
    - Removes punctuation
    - Converts to lowercase
3. **Negation handling**
    - Detects words like “not”
    - Reverses sentiment

Example:

Input: "This movie is amazing but bad"

amazing → +5

bad → -2

Final Score = +3 → Positive

Negation Example:

Input: "This movie is not bad"

bad → -2

not bad → +2

Final Score = +2 → Positive

📘 Strong OOP Design

- Abstract base class
- Runtime polymorphism

Making the system extensible

📘 Performance-Oriented Design

- unordered_map = fast lookup
- move semantics = faster execution
- DMA → memory efficiency

It is not just correct, but is optimized.

📘 Modular Architecture

Each part: Input, Processing, Scoring, is independent making it easy to upgrade later.

📘 Error-Resilient System

- File handling
- Exception safety

Makes it production-like.

📘 Educational Value

Our project demonstrates:

- NLP basics
- OOP principles
- STL usage
- Memory management

Making it a perfect academic integration.

LIMITATIONS

From real-world NLP limitations:

- Cannot detect sarcasm
- Cannot understand context
- Depends on dictionary quality
- Mixed sentiment sentences are hard

FUTURE IMPROVEMENTS

- Add machine learning models
- Use deep learning (BERT, etc.)
- Add GUI
- Support multi-language
- Improve dictionary